ADVANCED SOFTWARE ENGINEERING

Course Project Report · Semester 1

Do Vulnerability Detectors Memorize Names?

Cleaning the DiverseVul dataset and testing identifier renaming for deep-learning vulnerability detection on unseen projects

A reproduction and extension of Chen et al., "DiverseVul", RAID 2023

Submitted by

Name	Roll No.
Aashit Agrawal	26072005
Mohit Kukreja	26072016

Code and results: github.com/aashitagrawal/diversevul-vuln-detection

October 2026

Abstract

Deep learning models can be trained to flag C/C++ functions that contain security vulnerabilities, but recent work shows they fail badly on software projects they have not seen during training. The DiverseVul paper (Chen et al., RAID 2023) reports that a fine-tuned CodeT5 model drops from about 45% F1 on a random test split to about 9% F1 on unseen projects, and the authors suggest the model may be *"overfitting to patterns or coding idioms"* of the training projects. They also report that some labels in the dataset are wrong because security-fix commits sometimes change only whitespace.

This project reproduces and extends that work on a free GPU budget. **First**, we normalize every function (removing comments and whitespace) and find **4,397 groups of hidden duplicates**, including **900 groups whose identical code carries both labels**; this exposes **901 wrong "vulnerable" labels (4.8%)**, matching the paper's manual estimate of about 4%. We remove 5,440 functions to produce a cleaner dataset. **Second**, we fine-tune CodeT5-Small on the cleaned data and reproduce the generalization gap: **29.8% F1 on a random split versus 19.6% on 80 held-out projects**. **Third**, we test the authors' hypothesis directly by renaming every project-specific identifier to generic placeholders (`VAR_1`, `FUNC_1`). Renaming does **not** close the gap: unseen-project F1 changes only from 19.6% to 20.6%, while the false positive rate doubles (5.6% to 11.4%), and on the random split renaming lowers F1 from 29.8% to 23.7%. Identifier names therefore carry useful meaning rather than being mere memorization cues, and the cause of poor generalization must lie elsewhere.

Beyond the findings, the report documents the software engineering side of the work: a modular, reproducible pipeline under version control, a hybrid laptop + cloud GPU workflow, and the practical problems we diagnosed along the way.

Keywords: *vulnerability detection, DiverseVul, CodeT5, data quality, label noise, generalization, identifier renaming, reproducibility*

Contents

1. Introduction 5
1.1 Motivation 5
1.2 The paper we build on 5
1.3 Research questions 5
1.4 Contributions 5
1.5 Structure of this report 6
2. Background and Related Work 6
2.1 Vulnerabilities and CWE 6
2.2 Vulnerability datasets 6
2.3 Models for vulnerability detection 6
2.4 Key results of the DiverseVul paper 7
2.5 Follow-up work 7
3. Dataset and Exploratory Analysis 7
3.1 Projects 7
3.2 Vulnerability types 8
3.3 Function length 9
3.4 Observations that shape our method 9
4. Methodology 9
4.1 Pipeline overview 9
4.2 Improvement A: normalization-based de-duplication 10
4.3 Train, validation and test splits 10
4.4 Model and training 11
4.5 Improvement B: identifier renaming 11
4.6 Experimental design 12
5. Software Engineering Practices 12
5.1 Repository structure 12
5.2 Version control workflow 13
5.3 Hybrid laptop + cloud GPU workflow 13
5.4 Reproducibility 13
5.5 Problems encountered and how we solved them 14
6. Results 14
6.1 RQ1: Hidden duplicates and label conflicts 14
6.2 RQ2: Reproducing the generalization gap 15
6.3 RQ3: Does identifier renaming help? 15
6.4 Training behaviour 17
7. Discussion 17
7.1 Why does renaming not help? 17

7.2 Practical meaning 18
7.3 Lessons for software engineering 18
8. Threats to Validity and Limitations 18
8.1 Internal validity 18
8.2 External validity 18
8.3 Construct validity 18
8.4 Comparison with the paper 19
9. Future Work 19
10. Conclusion 19
References 20
Appendix A: How to Reproduce 21
Appendix B: Run Artifacts 21

1. Introduction

1.1 Motivation

Software vulnerabilities such as buffer overflows, use-after-free bugs and missing input checks are the root cause of many security incidents. Finding them early is valuable, but traditional static analyzers produce many false alarms and dynamic testing only explores the paths it happens to execute. Because deep learning has been very successful on text and code, researchers have asked whether a neural network can simply *read* a function and predict whether it is vulnerable.

For this to work, models need large, accurate datasets of vulnerable and non-vulnerable code, and they need to work on **new** software, not only on the projects they were trained on. The paper we build on, DiverseVul [1], addresses the first need and raises serious doubts about the second.

1.2 The paper we build on

Chen et al. collected **18,945 vulnerable and 330,492 non-vulnerable C/C++ functions** from 7,514 security-fix commits across 797 projects, and used them to compare 11 deep learning models. Their main conclusions are:

- Large language models pre-trained on code (CodeT5, NatGen) work best, reaching about **45–47% F1** on a random test split.
- On **projects never seen in training**, every model collapses, e.g. CodeT5-Small falls from **45.1% to 9.4% F1**. The authors write that the cause is unclear and suggest *"perhaps the model is overfitting to patterns or coding idioms that are specific to the particular projects"*, but they do not test this.
- Labels are noisy: in a manual check only about **60%** of "vulnerable" labels were correct, and about **4%** were wrong because the fixing commit only changed whitespace. The authors note that normalizing whitespace before de-duplication *"could slightly improve label accuracy"*, but leave this for future work.

1.3 Research questions

We turn the paper's open ends into three research questions:

1. **RQ1 (data quality):** How many functions in DiverseVul are duplicates once comments and whitespace are ignored, and how many of them carry contradictory labels?
2. **RQ2 (reproduction):** With a free cloud GPU, can we reproduce the gap between random-split and unseen-project performance?
3. **RQ3 (hypothesis test):** If we remove project-specific identifier names, does the model generalize better to unseen projects?

1.4 Contributions

- **A cleaned dataset** (325,052 functions) produced by a simple, explainable normalization and de-duplication step, which removes 900 label-conflicting groups. This implements the improvement the paper's authors proposed but did not carry out.
- **A reproduction** of the CodeT5-Small baseline on two splits (random and unseen projects) under a limited compute budget, with all executed notebooks kept as evidence.
- **A direct test** of the paper's memorization hypothesis via identifier renaming, in a 2 × 2 experiment (split × renaming).

- **A documented, reproducible software pipeline** with version control, and a record of the engineering problems we diagnosed and fixed.

1.5 Structure of this report

Section 2 summarizes background and related work. Section 3 explores the dataset. Section 4 describes our method, and Section 5 the software engineering practices behind it. Section 6 presents results, Section 7 discusses them, Section 8 lists threats to validity, Section 9 suggests future work, and Section 10 concludes.

2. Background and Related Work

2.1 Vulnerabilities and CWE

A **vulnerability** is a weakness in code that an attacker can exploit. The **Common Weakness Enumeration (CWE)** [11] gives each kind of weakness an identifier, e.g. CWE-787 *Out-of-bounds Write*, CWE-125 *Out-of-bounds Read* or CWE-416 *Use After Free*. MITRE publishes a yearly list of the 25 most dangerous CWEs; memory-safety weaknesses in C/C++ dominate it, which is why most vulnerability datasets focus on these languages.

2.2 Vulnerability datasets

Prior datasets differ mainly in **how they obtain labels**:

- **Synthetic** test suites such as Juliet/SARD contain hand-written vulnerable programs. Labels are accurate, but the code is small and unrealistic.
- **Static-analyzer labels** (Draper, D2A) mark code as vulnerable when an analyzer warns about it. These labels inherit the analyzers' high error rates.
- **Manual labels** (Devign [4]) are accurate but expensive: about 600 person-hours for two projects.
- **Security-fix commits** (ReVeal [3], BigVul [5], CVEfixes [6], CrossVul [7], DiverseVul [1]) find commits that fix a known vulnerability and label every function **changed** by the commit as vulnerable (before the fix) and the fixed version as non-vulnerable. This scales to real projects, but any unrelated change in the commit, even whitespace, produces a wrong label.

DiverseVul follows the last approach, crawling the security trackers snyk.io and bugzilla.redhat.com. It is the largest and most diverse of these datasets, covering 295 projects not present in any earlier one.

2.3 Models for vulnerability detection

Early work used LSTMs, CNNs and **graph neural networks** over code property graphs (e.g. ReVeal [3]). DiverseVul compares these with **transformer language models** from three families: RoBERTa (encoder-only), GPT-2 (decoder-only) and T5 (encoder-decoder [9]).

CodeT5 [2] is a T5 model pre-trained on source code with two identifier-aware objectives: predicting masked identifiers and tagging which tokens are identifiers. **NatGen** [12] continues pre-training CodeT5 by asking it to "naturalize" code that was transformed in meaning-preserving ways (for example, renamed variables or added dead code). In DiverseVul these code-specific pre-training objectives mattered more than model size: CodeT5-Small (60M parameters) matched models more than three times its size.

2.4 Key results of the DiverseVul paper

Table 1: Main findings of Chen et al. [1] relevant to this project.

Finding	Evidence in the paper
LLMs beat the GNN when data is large	Best LLM 47.2% F1 vs ReVeal 29.8% (merged data)
Code-specific pre-training matters most	CodeT5-Small (60M) ≈ CodeT5-Base / NatGen (220M)
More data gives diminishing returns	Adding DiverseVul helps 7 of 11 models, not CodeT5
Poor generalization to new projects	CodeT5-Small: 45.1% → 9.4% F1 on unseen projects
Class weights help a little	Unseen-project F1 9.4% → 17.2% with class-weighted loss
Labels are noisy	Only 60% of sampled vulnerable labels correct; ~4% whitespace-only

2.5 Follow-up work

The same research group later proposed **PrimeVul** [8], which uses stricter labeling and de-duplication and evaluates models more realistically. It found that earlier benchmarks overestimated model performance, which is consistent with our observation that cleaner data and stricter splits lower the scores.

3. Dataset and Exploratory Analysis

We downloaded the DiverseVul release (`diversevul_20230702.json`, 737 MB) from the authors' Google Drive link. It is a JSON-Lines file with one function per line and the fields `func` (source code), `target` (1 = vulnerable, 0 = not vulnerable), `cwe` (list of CWE ids), `project`, `commit_id`, `hash`, `size` and `message`. The script `src/explore.py` produces the statistics below.

Table 2: Dataset statistics compared with the paper. Small differences in projects and commits probably come from how the authors counted them (e.g. merged repository names).

Statistic	Our count	Paper
Functions	330,492	330,492
Vulnerable functions	18,945 (5.7%)	18,945
Non-vulnerable functions	311,547	311,547
Projects	800	797
Commits	7,653	7,514
Vulnerable functions without CWE	2,836	2,836 (18,945 − 16,109)
Exact duplicate functions	0	removed by MD5

3.1 Projects

The dataset is dominated by a few large projects. The Linux kernel alone contributes **72,270 functions (21.9%)**, and together with the older `linux-2.6` repository about a quarter of the data. This matters for the unseen-project split: whether Linux lands in the training or the test set changes the result considerably.

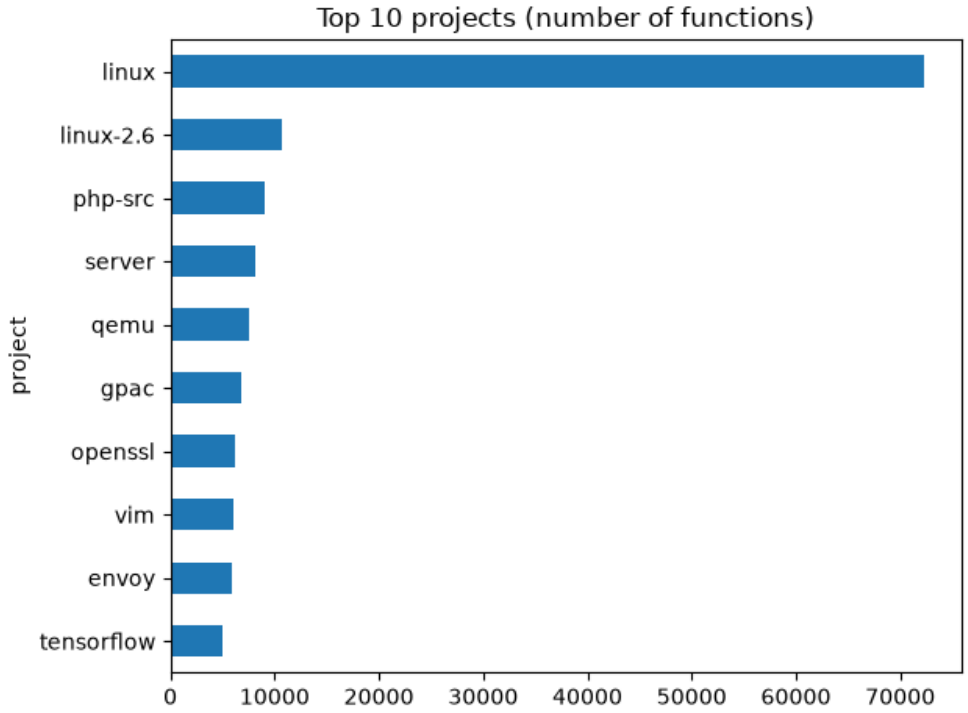


Figure 1: *The 10 projects with the most functions.*

3.2 Vulnerability types

Memory-safety weaknesses dominate. The top three, CWE-787 (out-of-bounds write), CWE-125 (out-of-bounds read) and CWE-119 (buffer errors), account for 31% of the 20,806 CWE labels, which cover 150 different CWEs. These numbers are identical to those in Table 1 of the paper.

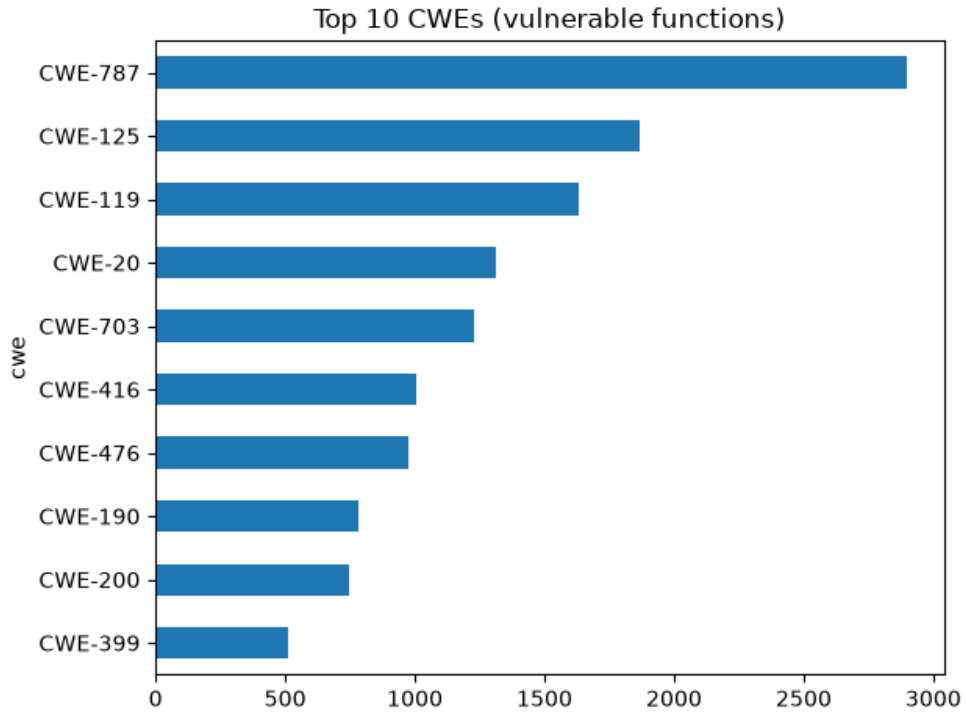


Figure 2: *The 10 most frequent CWEs among vulnerable functions.*

3.3 Function length

Vulnerable functions are much longer than non-vulnerable ones: a **median of 50 lines versus 18**, and a mean of 115 versus 39 lines. This has two consequences. First, length alone is a predictive signal, so a model may learn the shortcut "long function = vulnerable" instead of the actual bug. Second, models with a limited input length must cut long functions, and the vulnerable line may be lost.

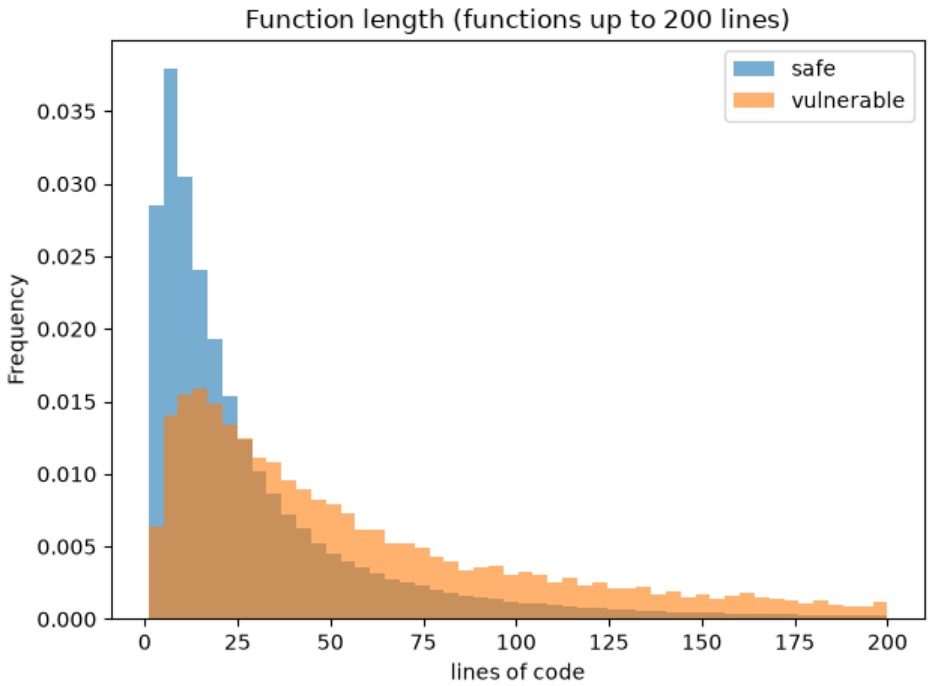


Figure 3: *Distribution of function length (functions up to 200 lines), normalized per class.*

3.4 Observations that shape our method

- **Severe class imbalance (5.7% vulnerable).** A model that always answers "safe" is 94% accurate, so we report **precision, recall, F1 and false positive rate** rather than accuracy.
- **No exact duplicates, but duplicates may hide behind formatting.** This motivates Improvement A (Section 4.2).
- **Long vulnerable functions** make input truncation a likely limitation of any model with a fixed input size.

4. Methodology

4.1 Pipeline overview

Figure 4 shows the full pipeline. Data preparation runs on a laptop CPU. The experiments run on a free Google Colab T4 GPU, where the notebook clones our GitHub repository and re-runs the same preparation scripts, so both environments use identical code.

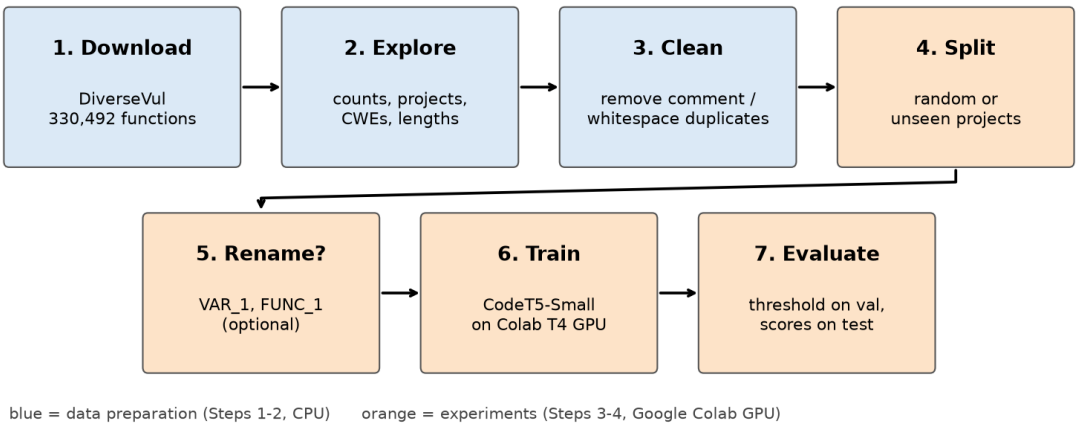


Figure 4: *Project pipeline. Steps 4–7 are executed once for each of the four experimental runs.*

4.2 Improvement A: normalization-based de-duplication

The authors de-duplicated functions by the MD5 hash of their **raw** text, so two functions that differ only in a space, a tab or a comment count as different. When a security-fix commit only re-formats a function, the version before the commit is labelled vulnerable and the version after it non-vulnerable, although the code is the same. Our script `src/clean_dedup.py` finds these cases in four steps:

1. **Normalize** each function: remove block comments (`/* ... */`) and line comments (`// ...`) with regular expressions, then delete all whitespace.
2. **Group** functions with identical normalized text.
3. **Drop conflicting groups:** if a group contains both labels, every function in it is removed, because we cannot tell which label is right.
4. **Keep one copy** of every remaining group and save the result as `data/diversevul_clean.json`.

We chose to drop conflicting groups instead of guessing a label. Guessing would itself introduce label noise, and dropping costs less than 0.6% of the data. Normalized text is used only for comparison; the saved functions keep their original formatting.

4.3 Train, validation and test splits

Following the paper, we evaluate two settings (`src/splits.py`, random seed 42):

- **Random split:** all functions are shuffled and divided 80% / 10% / 10%. Test functions come from the same projects as the training functions.
- **Unseen-project split:** 10% of the projects (**80 of 800**) are held out as the test set, so the model never sees any code from them. The remaining projects are shuffled into 90% training and 10% validation.

To fit training into free GPU time, we **down-sample non-vulnerable training functions** to three per vulnerable function (ratio 1 : 3, instead of about 1 : 17). All vulnerable training functions are kept. Validation and test sets are **not** changed, so all reported scores reflect the real class balance.

Table 3: Sizes of the two experimental splits on the cleaned dataset.

Split	Training (down-sampled)	Validation	Test
Random	57,232 (14,308 vuln.)	32,505 (1,836 vuln.)	32,506 (1,857 vuln.)
Unseen projects	60,092 (15,023 vuln.)	30,424 (1,724 vuln.)	20,813 (1,254 vuln.), 80 projects

4.4 Model and training

We fine-tune **CodeT5-Small** (`Salesforce/codet5-small`, 60M parameters), the smallest of the paper's best-performing models, with the Hugging Face Transformers library [10]. It is loaded as `T5ForSequenceClassification`: the full encoder-decoder processes the function, the hidden state at the end-of-sequence token is taken as a summary of the function, and a small classification head outputs two scores (vulnerable / not vulnerable).

Table 4: Training configuration, identical for all four runs.

Setting	Value	Reason
Max. input length	256 tokens	Training time (512 tokens took ~1 h 45 min per run)
Epochs	3	Free Colab GPU time
Learning rate	1×10^{-4}	5×10^{-5} under-fitted (val. F1 ≈ 0.28)
Batch size	16 (train), 64 (eval.)	Fits T4 memory
Precision	FP16 mixed precision	≈2× faster on T4
Optimizer / schedule	AdamW, linear decay, no warm-up	Library defaults
Model selection	Best epoch by validation F1	`load_best_model_at_end`
Class balance (train)	1 vulnerable : 3 non-vulnerable	Down-sampling (Section 4.3)
Hardware	NVIDIA T4 (Google Colab free tier)	≈50 min training per run

Choosing the decision threshold

Because training uses a 1 : 3 class ratio but real data is about 1 : 17, the usual rule "vulnerable if probability > 0.5" is miscalibrated. After training, we try every threshold from 0.05 to 0.95 in steps of 0.05 on the **validation** set, keep the one with the highest F1, and apply it unchanged to the **test** set. The test set is never used for any decision.

Evaluation metrics

With TP, FP, TN and FN the true/false positives/negatives (positive = vulnerable):

- **Precision** = TP / (TP + FP): of the functions we flag, how many are really vulnerable.
- **Recall** = TP / (TP + FN): of the vulnerable functions, how many we find.
- **F1** = 2 · Precision · Recall / (Precision + Recall): the main metric, as in the paper.
- **False positive rate (FPR)** = FP / (FP + TN): the share of safe functions wrongly flagged, which determines how many false alarms a developer must read.

4.5 Improvement B: identifier renaming

To test whether models fail on new projects because they memorize **project-specific names**, `src/rename_ids.py` replaces identifiers with generic placeholders before training and testing. A single regular expression scans each function for comments, string literals, character literals and identifiers, in that order:

- **Comments** are removed. **String and character literals** are kept unchanged.
- **C/C++ keywords** (`if`, `for`, `struct`, `sizeof`, ...) and about 70 **common library names** are kept. This includes the memory functions where bugs typically occur (`memcpy`, `strcpy`, `malloc`, `free`, `kmalloc`, `copy_from_user`, ...), `NULL` and fixed-width integer types.

- Every other identifier becomes `FUNC_n` if it is followed by `(`, otherwise `VAR_n`. The same name always maps to the same placeholder within one function, so the data flow is preserved.

```
// before
static int png_read_chunk(png_structp png_ptr, char *buf, size_t len)
{
    /* copy the chunk */
    char tmp[MAX_CHUNK];
    if (len > MAX_CHUNK)
        return png_error(png_ptr, "chunk too big");
    memcpy(tmp, buf, len);
    return 0;
}

// after
static int FUNC_1(VAR_1 VAR_2, char *VAR_3, size_t VAR_4)
{
    char VAR_5[VAR_6];
    if (VAR_4 > VAR_6)
        return FUNC_2(VAR_2, "chunk too big");
    memcpy(VAR_5, VAR_3, VAR_4);
    return 0;
}
```

Renaming all 325,052 functions takes about two minutes on a laptop CPU.

4.6 Experimental design

The two improvements combine into a 2 × 2 design. Each cell is one training run with identical hyper-parameters:

Table 5: The four experimental runs.

	Normal identifiers	Renamed identifiers
Random split (seen projects)	Run 1	Run 3
Unseen projects	Run 2	Run 4 (the key experiment)

If the memorization hypothesis is true, **Run 4 should clearly beat Run 2**. Run 3 tells us whether renaming harms the model on familiar code.

5. Software Engineering Practices

Although the project is a machine learning study, most of the effort was software engineering: making the work reproducible, splitting it into small explainable parts, and diagnosing failures across two very different environments.

5.1 Repository structure

Each step of the pipeline is a short, single-purpose Python script (20–115 lines) that can be run and explained on its own:

Table 6: Repository layout.

File	Purpose
`src/download_data.py`	Downloads DiverseVul from the authors' Google Drive (gdown)
`src/load_data.py`	Shared loader; reads the JSON-Lines file line by line
`src/explore.py`	Step 1: statistics and charts
`src/clean_dedup.py`	Step 2 / Improvement A: normalization and de-duplication
`src/splits.py`	Random and unseen-project splits, down-sampling
`src/rename_ids.py`	Step 4 / Improvement B: identifier renaming
`notebooks/train_codet5.ipynb`	Steps 3–4: training and evaluation on Colab
`notebooks/runs/run1-4_*.ipynb`	The four executed notebooks with all outputs (evidence)
`src/plot_results.py`, `src/report_figures.py`	Result table and figures
`results/`	Charts, cleaning summary and per-run scores (JSON)

5.2 Version control workflow

The project lives in a Git repository on GitHub. Steps 1–4 were each developed on a **feature branch** and merged through a **pull request** (PRs #1–#4) with a description of what changed, why, and what reviewers should check. Once the pipeline was in place, fixes and results were committed directly to `main`. Every commit message explains the reason for the change, so the history doubles as a log of the project. Large or derived files (the 737 MB dataset, trained model weights) are excluded with `.gitignore` and can be regenerated by the scripts.

5.3 Hybrid laptop + cloud GPU workflow

Neither team member has a GPU, so we separated **where code is written** from **where it runs**:

1. Code is written and tested on the laptop (CPU-only steps run there completely).
2. It is pushed to GitHub.
3. The Colab notebook clones the repository, so the cloud always runs the committed version of the code.
4. After each run, the executed notebook, including every printed output and the training table, is downloaded and committed under `notebooks/runs/` as **evidence** of the result.

5.4 Reproducibility

- Fixed random seed (42) for all splits and sampling; the training library also uses seed 42 by default.
- The dataset is downloaded and cleaned by scripts, never edited by hand.
- Library versions that caused failures are pinned (`transformers>=4.46,<5`).
- Hyper-parameters are kept in one settings cell, and only two switches (`SPLIT`, `RENAME_IDS`) change between runs.
- The scores of each run are stored as JSON in `results/`, and all tables and figures are generated from them by scripts.

5.5 Problems encountered and how we solved them

Table 7: Engineering problems diagnosed during the project.

Problem	Root cause	Fix
`pandas.read_json` fails: "Value is too big"	The `hash` field holds 128-bit integers	Parse line by line with Python's `json` module (`load_data.py`)
Tokenizer fails to load: "Input must be a List[Union[str, AddedToken]]"	Colab ships Transformers 5.x, which cannot load the CodeT5 tokenizer	Pin `transformers<5`
Risk of running out of memory during evaluation	The T5 classifier also returns encoder hidden states for every function	Keep only the logits (`preprocess_logits_for_metrics`)
"All examples must have the same number of <eos> tokens"	Some C code contains the literal text `</s>`, CodeT5's end-of-sequence token	Remove `</s>` from the code before tokenizing
`ImportError` inside Transformers after reloading Colab	Library files on disk changed after the old version was imported	Restart the Python session before training
Model barely learned (val. F1 0.28, loss 0.48 → 0.45)	Learning rate too low for 2–3 epochs	Raise learning rate from 5e-5 to 1e-4
One run would take ~1 h 45 min	512-token inputs on a T4	256 tokens, ~50 min per run
Run 3 lost mid-training	Colab free-tier GPU usage limit	Repeated the run the next day; save results after every run

6. Results

6.1 RQ1: Hidden duplicates and label conflicts

Table 8: Result of normalization-based de-duplication (`results/step2_dedup_summary.txt`).

Measure	Count
Functions before cleaning	330,492
Groups of duplicates after normalization	4,397
Extra copies inside those groups	4,540
Groups containing both labels (conflicts)	900 (1,815 functions)
…of which labelled vulnerable	901 (4.8% of all vulnerable functions)
Functions removed in total	5,440
Functions after cleaning	325,052 (18,001 vulnerable, 307,051 safe)

Although the dataset contains **no exact duplicates**, ignoring comments and whitespace reveals 4,397 duplicate groups. In 900 of them, **the same code is labelled both vulnerable and not vulnerable**. One example comes from the `yara` project: the function `input()` of its generated lexer appears twice with identical code, once with label 1 and once with label 0.

> **Answer to RQ1.** About **4.8% of the vulnerable labels** (901 functions) belong to code that also appears with the opposite label, so they are almost certainly wrong. This automatically confirms the paper's manual estimate of about 4% whitespace-only label errors, and the fix is cheap: one normalization step before de-duplication.

6.2 RQ2: Reproducing the generalization gap

Table 9: CodeT5-Small test results (%). Paper numbers: Tables 4 and 5 of [1], trained on the merged "Previous + DiverseVul" data.

Setting	Precision	Recall	F1	FPR
Paper: random split	48.41	42.22	45.10	3.88
Paper: unseen projects	13.35	7.24	9.39	1.78
Ours, Run 1: random split	24.17	38.99	29.84	7.41
Ours, Run 2: unseen projects	18.94	20.26	19.58	5.56

We reproduce the direction of the paper's result: **F1 falls by about a third (29.8 → 19.6)** on projects the model has never seen, and recall nearly halves (39.0 → 20.3). In absolute numbers, on the random test set the model finds **724 of 1,857** vulnerable functions while raising about **2,270 false alarms** among 30,649 safe ones. On the unseen projects it finds **254 of 1,254** with about **1,090 false alarms**.

Our random-split F1 is lower than the paper's (29.8 vs 45.1). Section 8 discusses why: shorter inputs, much less training data, and cleaner data. Our unseen-project F1, on the other hand, is higher (19.6 vs 9.4). Possible reasons are that we tune the decision threshold on validation data (the paper does not describe threshold tuning), that our 80 held-out projects are a different random sample, and that the paper trained on a different mix of datasets; we did not test which of these matters most. So the relative gap is smaller than in the paper, but clearly present.

> **Answer to RQ2.** Yes. Even with a small budget, the model generalizes clearly worse to unseen projects (F1 −10.3 points, −34%).

6.3 RQ3: Does identifier renaming help?

Table 10: Test results of all four runs (%).

Run	Test data	Identifiers	Threshold	Precision	Recall	F1	FPR
1	random	normal	0.60	24.17	38.99	29.84	7.41
2	unseen	normal	0.65	18.94	20.26	19.58	5.56
3	random	renamed	0.50	20.39	28.33	23.71	6.70
4	unseen	renamed	0.40	15.19	31.74	20.55	11.36

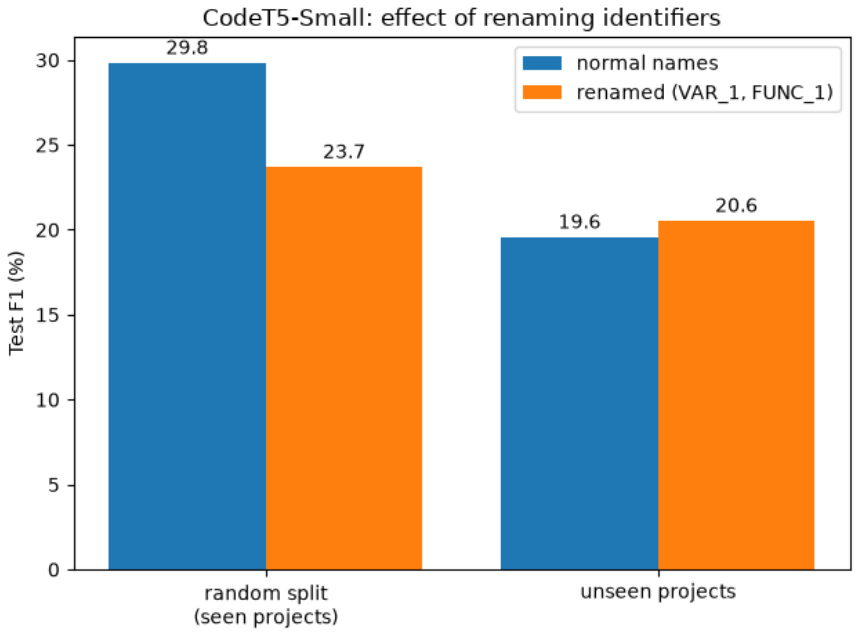


Figure 5: *Test F1 with normal and renamed identifiers on both splits.*

Renaming has two effects:

- **On the random split it clearly hurts:** F1 drops by 6.1 points (29.8 → 23.7), with both precision and recall falling.
- **On unseen projects it barely changes F1:** +1.0 point (19.6 → 20.6). With a single run per setting, this is within normal run-to-run variation. Renaming does change the model's behaviour, though: recall rises from 20.3% to 31.7% (about 400 instead of 254 vulnerabilities found), but the false positive rate **doubles** from 5.6% to 11.4% (about 2,220 instead of 1,090 false alarms).

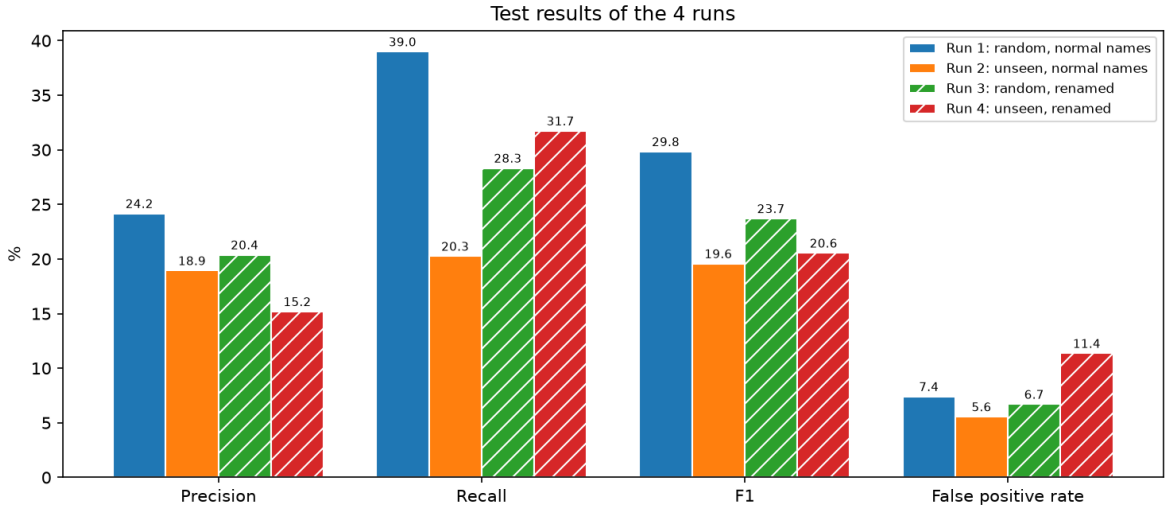


Figure 6: *Precision, recall, F1 and false positive rate of the four runs. Hatched bars use renamed identifiers.*

> **Answer to RQ3.** No. Removing project-specific names does **not** meaningfully improve generalization to unseen projects. It only trades precision for recall, and it costs 6 F1 points on familiar projects. The paper's hypothesis that models fail because they memorize project-specific names is **not supported** by our experiment.

6.4 Training behaviour

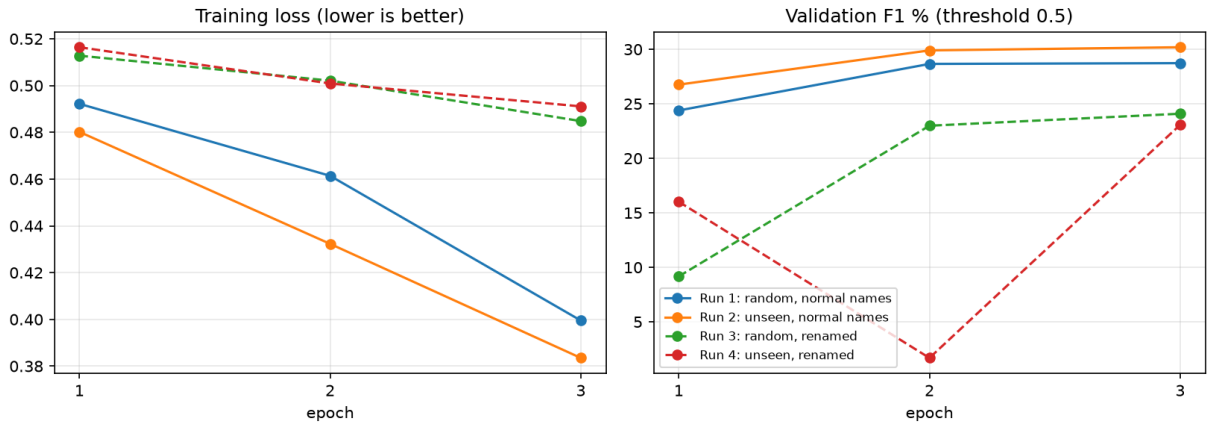


Figure 7: *Training loss and validation F1 per epoch (validation F1 uses the default 0.5 threshold).*

The training curves in Figure 7 add three observations:

- **Renamed code is harder to learn.** The training loss of Runs 3 and 4 stays at 0.48–0.49 after three epochs, compared with 0.38–0.40 with normal names. Without names, the model has less information to fit.
- **Training with renamed code is unstable.** Run 4's validation F1 collapses to 1.7% in epoch 2 before recovering to 23.1%, so the model temporarily predicted almost everything as safe.
- **Validation scores hide the generalization problem.** Run 2 has the *highest* validation F1 (30.2%), because its validation set comes from the seen projects, yet its test F1 on unseen projects is only 19.6%. A model selected on an in-distribution validation set looks better than it really is.

7. Discussion

7.1 Why does renaming not help?

Our hypothesis was that names like `png_ptr` or `yyg->yy_c_buf_p` let the model recognize *which project* a function comes from, and that removing them would force it to learn project-independent bug patterns. The results suggest a different picture:

- **Names carry meaning, not just identity.** Names such as `len`, `size`, `buf`, `count`, `offset` or `free_buffer` describe what a value is and how it is used. A missing bounds check on `len` is easier to spot when the model knows it is a length. Removing names removes this semantic information, which explains the 6-point drop on the random split.
- **CodeT5 was pre-trained to use identifiers.** Its pre-training explicitly predicts and tags identifiers [2]. Placeholders such as `VAR_17` are far from anything it saw during pre-training, so renaming partly disables what makes CodeT5 strong.
- **Project-specific cues are not only names.** Coding style, macro usage, typical function structure and function length all differ between projects and survive renaming. If the model relies on these, renaming cannot remove the shortcut.
- **The threshold does not transfer.** The threshold is tuned on validation data from seen projects. On unseen projects the renamed model's probabilities are distributed differently, so the same threshold produces many more false alarms. This is a calibration problem rather than a representation problem.

7.2 Practical meaning

Even our best run flags about 7% of safe functions as vulnerable. In a project with 10,000 functions, a developer would have to review about **700 false alarms** to find a few hundred real bugs. Like the original authors, we conclude that deep learning vulnerability detectors of this size are **not yet practical** as stand-alone tools, especially on new code bases.

7.3 Lessons for software engineering

- **Data quality is a software problem.** A one-line normalization step exposed hundreds of wrong labels. Datasets need the same testing and review as code.
- **Evaluate the way the tool will be used.** A random split overstates performance. A tool that will run on *new* projects must be evaluated on held-out projects, and model selection should not rely only on in-distribution validation data.
- **Environment drift breaks ML code.** Two of our failures came from library version changes in a managed cloud environment. Pinning versions and committing executed notebooks made the results reproducible and verifiable.

8. Threats to Validity and Limitations

8.1 Internal validity

- **Single run per setting.** Each configuration was trained once (seed 42). Differences of about 1 F1 point, such as Run 4 vs Run 2, may be noise. The 6-point and 10-point differences are much larger and more reliable.
- **Different thresholds per run.** Each run uses the threshold that was best on its own validation set (0.40–0.65). This is the fair procedure, but it means precision/recall trade-offs differ between runs; F1 is the most comparable number.
- **Truncation to 256 tokens.** Vulnerable functions are long (median 50 lines), so the vulnerable line is often cut off. This likely lowers all our scores.
- **Down-sampled training data.** We train on about 57,000–60,000 functions instead of the full training set, and for only 3 epochs.

8.2 External validity

- We study **one model** (CodeT5-Small), **one dataset** (DiverseVul only; the paper merged five more), and **one language family** (C/C++). Larger models or other datasets might react to renaming differently.
- The unseen-project result depends on **which 80 projects** were held out. A different random choice, in particular one including Linux, could give different numbers.

8.3 Construct validity

- Our cleaning removes only **one type** of label error (formatting-only changes). The paper estimates that only about 60% of vulnerable labels are correct, so most label noise remains, and it affects both training and evaluation.
- Both normalization and renaming use **regular expressions, not a real C parser**. Rare cases (comment markers inside strings, macros, type names such as `png_structp` treated as variables) are handled approximately.

8.4 Comparison with the paper

Our numbers are not directly comparable with the paper's, because the paper trained on the merged "Previous + DiverseVul" dataset (about 419,000 training functions), with longer inputs and full training. We compare **trends** (the size of the gap, the effect of renaming), not absolute scores.

9. Future Work

- **Repeat each run with several seeds** and report mean and spread, to decide whether small differences are real.
- **Partial renaming:** remove only project-specific *prefixes* (`png_`, `gnutls_`, `yy`) while keeping meaningful words (`len`, `buf`, `size`). This would separate "project identity" from "meaning" more precisely than full renaming.
- **Longer inputs and full data** (512 tokens, all training functions, more epochs) on a stronger GPU, to get closer to the paper's absolute numbers.
- **Better calibration for unseen projects**, e.g. choosing the threshold on a validation set made of held-out projects, or combining renaming with class-weighted loss, which the paper found helpful.
- **Stricter datasets** such as PrimeVul [8], and larger code models such as CodeT5+ or UniXcoder.
- **Tool integration:** wrap the trained model in a command-line tool that scores each function of a C file, to study usability with real developers.

10. Conclusion

We reproduced and extended the DiverseVul study of deep-learning vulnerability detection using only free computing resources. A simple normalization step found **901 contradictory vulnerable labels (4.8%)** that the original de-duplication missed, which automatically confirms the authors' manual estimate. Fine-tuning CodeT5-Small reproduced the central problem of the field: performance drops by about a third (**29.8% → 19.6% F1**) on projects the model has never seen.

We then tested the authors' explanation that models memorize project-specific idioms by renaming all identifiers to generic placeholders. Renaming **did not close the gap**: it left unseen-project F1 almost unchanged, doubled the false positive rate, and hurt performance on familiar projects. Identifier names turn out to be a useful source of meaning, not merely a memorization shortcut, so the real cause of poor generalization lies elsewhere and remains an open problem.

Finally, the project showed that careful data handling, honest evaluation splits and reproducible engineering practices are as important for machine learning on code as the choice of model.

References

[1] Y. Chen, Z. Ding, L. Alowain, X. Chen and D. Wagner, “DiverseVul: A New Vulnerable Source Code Dataset for Deep Learning Based Vulnerability Detection,” in Proc. 26th Int. Symposium on Research in Attacks, Intrusions and Defenses (RAID ’23), 2023. doi:10.1145/3607199.3607242

[2] Y. Wang, W. Wang, S. Joty and S. C. H. Hoi, “CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation,” in Proc. EMNLP, 2021.

[3] S. Chakraborty, R. Krishna, Y. Ding and B. Ray, “Deep Learning based Vulnerability Detection: Are We There Yet?,” IEEE Transactions on Software Engineering, 2021.

[4] Y. Zhou, S. Liu, J. Siow, X. Du and Y. Liu, “Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks,” in Proc. NeurIPS, 2019.

[5] J. Fan, Y. Li, S. Wang and T. N. Nguyen, “A C/C++ Code Vulnerability Dataset with Code Changes and CVE Summaries,” in Proc. MSR, 2020.

[6] G. Bhandari, A. Naseer and L. Moonen, “CVEfixes: Automated Collection of Vulnerabilities and Their Fixes from Open-Source Software,” in Proc. PROMISE, 2021.

[7] G. Nikitopoulos, K. Dritsa, P. Louridas and D. Mitropoulos, “CrossVul: A Cross-Language Vulnerability Dataset with Commit Data,” in Proc. ESEC/FSE, 2021.

[8] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray and Y. Chen, “Vulnerability Detection with Code Language Models: How Far Are We?,” in Proc. ICSE, 2025.

[9] C. Raffel et al., “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” Journal of Machine Learning Research, 21, 2020.

[10] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in Proc. EMNLP: System Demonstrations, 2020.

[11] MITRE, “Common Weakness Enumeration (CWE),” https://cwe.mitre.org

[12] S. Chakraborty, T. Ahmed, Y. Ding, P. Devanbu and B. Ray, “NatGen: Generative Pre-training by ‘Naturalizing’ Source Code,” in Proc. ESEC/FSE, 2022.

Appendix A: How to Reproduce

All commands are run from the repository root.

```
git clone https://github.com/aashitagrawal/diversevul-vuln-detection.git
cd diversevul-vuln-detection
pip install -r requirements.txt
python src/download_data.py   # download DiverseVul (~740 MB) into data/
python src/explore.py         # Step 1: statistics and charts
python src/clean_dedup.py     # Step 2: cleaned dataset + summary
python src/splits.py          # sizes of both splits
python src/rename_ids.py      # Step 4: renaming example
```

Training (Steps 3–4) runs on Google Colab: open `notebooks/train_codet5.ipynb`, select a T4 GPU and run all cells. Set `SPLIT` (`"random"` / `"unseen"`) and `RENAME_IDS` (`False` / `True`) in the settings cell for the four runs. When moving to the next run, use *Runtime → Run cell and below* from the settings cell. Afterwards:

```
python src/plot_results.py    # result table + results/f1_comparison.png
python src/report_figures.py  # training curves, metric comparison, pipeline
```

Appendix B: Run Artifacts

Table 11: Evidence for each training run, stored in the repository.

Run	Executed notebook	Scores
1	`notebooks/runs/run1_random_baseline.ipynb`	`results/random_baseline.json`
2	`notebooks/runs/run2_unseen_baseline.ipynb`	`results/unseen_baseline.json`
3	`notebooks/runs/run3_random_renamed.ipynb`	`results/random_renamed.json`
4	`notebooks/runs/run4_unseen_renamed.ipynb`	`results/unseen_renamed.json`

Table 12: Training loss / validation F1 (%) per epoch, copied from the executed notebooks.

Epoch	Run 1	Run 2	Run 3	Run 4
1	0.492 / 24.4	0.480 / 26.8	0.513 / 9.2	0.516 / 16.0
2	0.461 / 28.7	0.432 / 29.9	0.502 / 23.0	0.501 / 1.7
3	0.400 / 28.7	0.384 / 30.2	0.485 / 24.1	0.491 / 23.1